COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr, Jimlord Quejado

ACTIVITY NO. 1 Algorithm, Flowchart and Pseudocode

1. PROCEDURE OUTPUT

```
#include <iostream>
#include "singly_ll.h"

int main(){
    //Creation of node in the stack
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //allocates memory in the heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;


    //Initializing of the data in the list
    head->data = 'G';
    second->data = 'I';
    third->data = 'A';
    fourth->data = 'N';
```

Image 1

```
//Link the list
head->next = second;
second->next = third;
third->next = fourth;

std::cout << "Testing travesal\n";

singleListTravesal(head);

std::cout << "\nTesting of sllinsertHead\n";
sllinsertHead('Q', &head);
singleListTravesal(head);

std::cout << "\nTesting of sllGeneralInsert\n";
sllGeneralInsert('A', head->next->next);
singleListTravesal(head);

std::cout << "\nTesting of sllInsertEnd\n";
sllInsertEnd('U', &head);
singleListTravesal(head);
```

Image 2

```
    std::cout << "\nTesting deleting node\n";
    sllDeleteNode('Q', &head);
    singleListTravesal(head);
    std::cout << std::endl;
    sllDeleteNode('A', &head);
    singleListTravesal(head);
    std::cout << std::endl;
    sllDeleteNode('U', &head);
    singleListTravesal(head);
    std::cout << std::endl;

    std::cout << "Deleting list\n";
    sllDeleteList(&head);
    singleListTravesal(head);

    return 0;
}
```

Image 3

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template<typename T>
class SingleList{
    public:
    T data; //contains data to be stored in the node
    SingleList* next = nullptr;
};

template<typename T>
void ListTravesal(SingleList<T>* head){
    if (head == nullptr){
        std::cout << "There is no list\n";
        return;
    }

    while(head != nullptr){
        std::cout << head->data;

        if (head->next != nullptr) {
            std::cout << "->";
        }

        head = head->next;
    }
    std::cout << std::endl;
}
```

Image 4

```cpp
template<typename T>
void sllinsertHead(T newData,SingleList<T>** currentHead){
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;

}

template <typename T>
void sllGeneralInsert(T newData,SingleList<T>* prevNode){
    //If check if head
    if (prevNode == nullptr){
        std::cout << "Previous value cannot be null\n";
        return;
    }
    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //Store Data
    newNode->data = newData;
    //Point new node
    newNode->next = prevNode->next;
    prevNode->next = newNode;

}
```

Image 5

```cpp
template<typename T>
void sllInsertEnd(T newData, SingleList<T>** head){
    SingleList<T>* newNode = new SingleList<T>;
    SingleList<T>* currentNode = *head;
    newNode->data = newData;
    newNode->next = nullptr;

    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;
}

template <typename T>
void sllDeleteNode(T findData,SingleList<T>** head){

    if (head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;


    while (currNode != nullptr &&  currNode->data != findData){
        prevNode = currNode;
        currNode = currNode ->next;
    }
```

Image 6

```
    if (currNode == nullptr){
        std::cout << "\nData Not Found\n";
        return;
    }

    if (prevNode == nullptr){
        *head = currNode->next;
    }
    else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}


template <typename T>
void sllDeleteList(SingleList<T>** head){
    SingleList<T>* current = *head;

    while (current != nullptr){
        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}
#endif
```

Image 7

Analysis

Image 1 to 3 shows the main function of the linked lists that I created. It basically just imports and prints the result of every function out of the separate H file that I created. As you can see in image 3, I created a class for an object that I will use. Next, I tried the ListTraversing function. It reads the file in order and prints a "->" sign in between the characters that it goes through until it's null. Next is the sllinserted. This adds a new node and another character to the start to loop into and print. In the general version, it basically just enables you to add a new node anywhere as long as it is not a null pointer. And the end version is just adding a node into the end. The slldeletenode deletes a node by using the pointer and after that, it turns it into a nullptr. This function basically allows you to delete any node that you want. And lastly is the delete list where it allows you to delete all the nodes.

Outputs

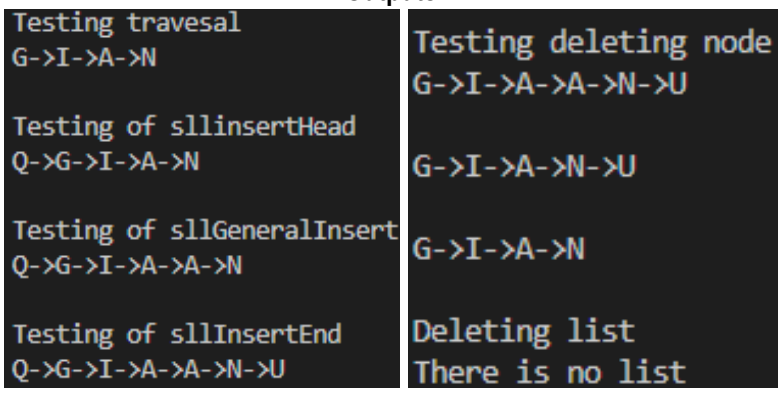


Image 8 and 9

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.